

Pradeep Jewellers — pj-shop

Pricing Module — Technical Documentation (v2)

Updated after single-function consolidation — June 2026

Purpose: Document every point where price is calculated or triggered, with exact file paths, distinguishing server-side execution from client-side triggers.

Architecture principle: ONE function calculates jewellery price on the server — `calculateVariantPrice()`. The browser never runs the formula directly.

1. Executive Summary

All jewellery prices use `calculateVariantPrice()` in `src/utils/calculateVariantPrice.ts`. Gold rate per karat is read from Supabase `dev.store_metal_prices`. Making charge percentage comes from Shopify product metafields (exposed as `product.makingChargePercent` in the catalog API). Weight comes from Shopify variant data. GST is 3% on gold value + making charge. Option adjustments (metal, carat, diamond, size `priceAdjustment` fields) are summed on the client and stored on cart lines.

Quick reference counts

Category	Count	Details
Single pricing function (server)	1	<code>calculateVariantPrice()</code> in <code>src/utils/calculateVariantPrice.ts</code> — all paths use this
Server execution entry points	5	API route, catalog list, catalog detail, add-to-cart, checkout revalidation
Client triggers of server calc (POST <code>/api/price/calculate</code>)	2 hooks	<code>useProductConfiguredPrice</code> (×2 consumers), <code>useCartLineBreakdowns</code> (fallback)
Client local math only	1	<code>sumOptionLineAmounts()</code> — option adjustments added on top of <code>finalPrice</code>
Display-only (no recalculation)	4+	Collection cards, search, recommended — show pre-baked <code>product.price</code>
Removed / no longer present	4	<code>computeListVariantFinalPrice</code> , <code>useProductBasePrice</code> , <code>mapSupabaseProductRow</code> , duplicate panel pricing

2. The Single Pricing Function

2.1 `calculateVariantPrice` — authoritative

- File: `src/utils/calculateVariantPrice.ts`
- Export: `calculateVariantPrice(input)` — throws `GoldPriceUnavailableError` if no DB gold rate
- Wrapper: `tryCalculateVariantPrice(input)` — returns null instead of throwing (used in catalog mapping)

Formula steps:

1. `parseKaratNumber(carat)` — `src/utils/karat.ts`
2. `getGoldPricingForKarat(karat)` — `src/utils/goldPrice.ts` → `dev.store_metal_prices` (30s cache)
3. `actualGoldPrice = weight × perGramRate`
4. `makingCharge = actualGoldPrice × resolveMakingChargeRate(makingChargePercent, makingChargeType)` — `src/utils/makingCharge.ts`, default 15%
5. `subtotal = actualGoldPrice + makingCharge`
6. `gst = subtotal × 0.03`
7. `finalPrice = Math.round(subtotal + gst)`

Inputs:

- weight — grams; normalized via resolveVariantWeight() in src/utils/resolveVariantWeight.ts (default 5g)
- carat — variant option label e.g. "22K Yellow Gold"
- makingChargePercent — from Shopify metafields via catalog mapping
- makingChargeType — optional Shopify making_charge_type metafield

2.2 What is NOT in calculateVariantPrice

- Option adjustments (priceAdjustment on metal/carat/diamond/size) — client-only via src/utils/priceBreakdownOptions.ts
- Diamond surcharges beyond option adjustments
- Shopify list/compare-at display prices when gold DB is empty (fallback only)

3. Data Sources

Gold rate (karat) — Database

- Table: dev.store_metal_prices (source = manual, metal = gold, unit = gram)
- Reader: src/utils/goldPrice.ts → getGoldPricingForKarat()
- Admin save: src/app/api/metal-prices/route.ts clears cache via clearGoldPriceCache()
- Display-only (not used in product calc): src/app/api/gold-rate/route.ts, src/utils/metalRatesFromDb.ts

Making charge — Shopify API

- Storefront metafields: custom.making_charge_type, custom.making_charge
- Parser: parseMakingChargeFromMetafields() in src/utils/makingCharge.ts
- Resolver in catalog: resolveShopifyMakingChargePercentFromStorefront() / resolveShopifyMakingChargePercent() in src/lib/shopify.ts
- Stored on Product as makingChargePercent — passed to calculateVariantPrice on every calc

Weight — Shopify variant API

- Shopify variant.weight + weightUnit converted to grams in src/lib/shopify.ts (shopifyWeightToGrams)

4. Server-Side Calculation Points (Detailed)

The server runs calculateVariantPrice (directly or via tryCalculateVariantPrice) at these entry points:

#	File	Function / Route	When it runs	Calls
S1	src/utils/calculateVariantPrice.ts	calculateVariantPrice()	Any server price need	getGoldPricingForKarat (DB) + resolveMakingChargeRate (Shopify %)
S2	src/app/api/price/calculate/route.ts	POST handler	Browser POST from client hooks	calculateVariantPrice()
S3	src/lib/shopify.ts	mapShopifyProductListItem()	GET /api/products (catalog list)	tryCalculateVariantPrice() per sample variant (up to 4)
S4	src/lib/shopify.ts	mapShopifyProduct()	GET /api/products/[slug] (product detail)	tryCalculateVariantPrice() per Shopify variant
S5	src/lib/serverCartPricing.ts	resolveTrustedCartUnitPrice()	POST /api/cart (add to cart)	Snapshot OR calculateVariantPrice() OR catalog variant.price
S6	src/lib/serverCartPricing.ts	revalidateCartLineUnitPrice()	POST /api/cart/checkout	parseBreakdownFromLine OR resolveTrustedCartUnitPrice
S7	src/app/api/cart/route.ts	POST	Add / update cart line	resolveCartLinePricing() → S5
S8	src/app/api/cart/checkout/route.ts	POST	Buy now or cart checkout	resolveCartLinePricing (buy now) or revalidateCartLineUnitPrice (cart)

4.1 Server count — how many times per user action

- Catalog list page load: S3 runs tryCalculateVariantPrice up to 4× per product (sample variants)
- Product detail API load: S4 runs tryCalculateVariantPrice 1× per Shopify variant
- Product page in browser: C1 triggers S2 once on load; again on each customization change
- Customize modal: C3 uses same C1 — S2 on each picker change
- Add to cart: S7 → S5 once per line (snapshot if breakdown sent; else live calculateVariantPrice)
- Cart checkout: S8 → S6 once per cart line
- Cart page breakdown: C4 may trigger S2 per line only when _pj_* snapshot attrs are missing

4.2 Server paths that do NOT recalculate

- GET /api/cart — reads _pj_custom_price_inr from Shopify line attributes
- trustedPriceFromStoredBreakdown() in serverCartPricing.ts — reads _pj_price_subtotal + _pj_gst + _pj_option_adj
- parseBreakdownFromLine() in src/lib/cartBreakdown.ts — reconstructs breakdown from stored attrs

4.3 Shopify fallback (no DB gold rates)

- tryCalculateVariantPrice returns null → mapShopifyProduct* uses raw Shopify variant.price.amount
- resolveTrustedCartUnitPrice falls back to catalogVariantUnitPrice() from pre-fetched product

5. Client-Side Points (Detailed)

The browser does NOT execute calculateVariantPrice. It either triggers S2 via fetch, sums option adjustments, or displays pre-baked prices.

#	File	Component / Hook	When it runs	Server call?
C1	src/hooks/useProductConfiguredPrice.ts	useProductConfiguredPrice	Product selections change	Yes — always POST /api/price/calculate → S2
C2	src/components/productComponent/ProductDetailClient.tsx	Product detail page	Page load + customize confirm	Via C1
C3	src/components/productComponent/ProductPurchasePanel.tsx	Customize modal	Picker changes in modal	Via C1 (shared hook — no duplicate logic)
C4	src/hooks/useCartLineBreakdowns.ts	Cart breakdown expand	Cart lines change	Only if no _pj_* snapshot — POST /api/price/calculate
C5	src/utils/priceBreakdownOptions.ts	sumOptionLineAmounts()	Inside C1	No — adds metal/diamond/size priceAdjustment to total
C6	src/components/productComponent/CollectionProductCard.tsx	Shop grid cards	List render	No — displays product.price baked at S3
C7	src/components/cart/CartPageClient.tsx	Cart totals	Cart page load	No for line prices — GET /api/cart reads stored _pj_custom_price_inr

5.1 Client count summary

- Full price calc on client: 0 — always delegated to POST /api/price/calculate
- API trigger hooks: 2 — useProductConfiguredPrice, useCartLineBreakdowns (conditional)
- Consumers of useProductConfiguredPrice: 2 — ProductDetailClient, ProductPurchasePanel
- Local arithmetic: sumOptionLineAmounts() inside useProductConfiguredPrice → totalPrice = finalPrice + optionAdjustments

6. End-to-End Flows

6.1 Product detail page

GET /api/products/[slug]

SERVER S4: mapShopifyProduct → tryCalculateVariantPrice per variant
→ product.price, variant.price, makingChargePercent from Shopify

Browser: ProductDetailClient

CLIENT C1: useProductConfiguredPrice

→ POST /api/price/calculate (SERVER S2)

CLIENT C5: + optionAdjustments → totalPrice displayed

6.2 Customize modal

ProductModal → ProductPurchasePanel

CLIENT C3: same useProductConfiguredPrice hook (no duplicate pricing code)

→ POST /api/price/calculate on each selection change

6.3 Add to cart

productCheckout.addProductToCart (src/lib/productCheckout.ts)

→ POST /api/cart (SERVER S7)

→ resolveCartLinePricing → resolveTrustedCartUnitPrice (S5)

→ buildLineAttributesForCart (src/lib/cartLinePayload.ts)

→ Shopify cart line with _pj_custom_price_inr + breakdown attrs

6.4 Checkout

POST /api/cart/checkout (SERVER S8)

→ revalidateCartLineUnitPrice per line (S6)

→ createDraftCheckoutFromLines (src/lib/shopify.ts) with priceOverride

7. Complete File Index

Core (single function)

- src/utils/calculateVariantPrice.ts — calculateVariantPrice, tryCalculateVariantPrice, VariantPriceBreakdown type
- src/utils/goldPrice.ts — DB gold rates by karat
- src/utils/makingCharge.ts — Shopify making % parsing
- src/utils/karat.ts — Karat label parsing
- src/utils/resolveVariantWeight.ts — Weight fallback
- src/utils/priceBreakdownOptions.ts — Option adjustment lines

Server — catalog & Shopify

- src/lib/shopify.ts — mapShopifyProductListItem, mapShopifyProduct, getProductByHandle
- src/lib/fetchSingleCatalogProduct.ts — Product fetch wrapper
- src/app/api/products/route.ts — List endpoint
- src/app/api/products/[slug]/route.ts — Detail endpoint

Server — price API & cart

- src/app/api/price/calculate/route.ts
- src/lib/serverCartPricing.ts

- src/lib/cartBreakdown.ts
- src/lib/cartLinePayload.ts
- src/lib/priceCalculateResponse.ts
- src/app/api/cart/route.ts
- src/app/api/cart/checkout/route.ts

Client — hooks & UI

- src/hooks/useProductConfiguredPrice.ts — shared pricing hook
- src/hooks/useCartLineBreakdowns.ts — cart breakdown display
- src/components/productComponent/ProductDetailClient.tsx
- src/components/productComponent/ProductPurchasePanel.tsx
- src/components/productComponent/PriceCalculationBreakdown.tsx
- src/components/cart/CartPageClient.tsx
- src/lib/productCheckout.ts — passes price/breakdown to cart APIs

8. Removed in v2 Consolidation

- src/lib/shopify.ts — computeListVariantFinalPrice() — DELETED (was duplicate formula)
- src/lib/shopify.ts — getListKaratPricing() / listKaratPricingCache — DELETED
- src/hooks/useProductBasePrice.ts — DELETED (was unused)
- src/lib/mapSupabaseProductRow.ts — DELETED (was unused)
- ProductPurchasePanel inline /api/price/calculate useEffect — REMOVED (now uses shared hook)
- useProductConfiguredPrice variant snapshot shortcut — REMOVED (always calls API for fresh DB gold rate)

9. Design Rules

- One function: calculateVariantPrice() is the only place the formula lives
- Server wins: cart/checkout uses resolveTrustedCartUnitPrice, not client customPrice alone
- Snapshots: _pj_* cart attributes lock price at add-to-cart to avoid gold-rate drift
- Karat from DB: getGoldPricingForKarat reads live manual rates per purity label (22K, 18K, etc.)
- Making % from Shopify: product.makingChargePercent set during mapShopifyProduct* from metafields
- GST: fixed 3% inside calculateVariantPrice only

Legacy routes (not primary flow)

- src/app/api/checkout/route.ts — Storefront checkout at Shopify variant price
- src/app/api/checkout/cart/route.ts — older path bypassing serverCartPricing

— End of document (v2) —